S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR.

Practical No. 10

Aim: Apply prompt engineering to construct a prompt for learning the concept of Adversarial Search.

Name of Student: Shivani Dhakate

Roll No.: CS23109

Semester/Year: V/III

Academic Session: 2025-2026

Date of Performance:

Date of Submission:

AIM: To design and refine an effective prompt using prompt engineering techniques for learning the concept of Adversarial Search from Artificial Intelligence.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- To be able to understand the concept of a *prompt* (input given to an AI model).
- To be able to understand the concept of *prompt engineering* (designing inputs to get desired outputs)
- To be able to work with basic idea and models like ChatGPT.
- To be able to distinguish between a vague prompt and a well-structured one.

THEORY:

i. Fundamentals of Prompt Engineering

Prompt engineering is the process of designing and structuring inputs (prompts) to get accurate, relevant, and well-formatted outputs from AI systems such as ChatGPT. Prompt engineering is how you ask a question to AI so that it gives the best possible answer.

ii. Characteristics of a Good Prompt

A good prompt is based on the following core principles:

- **Clarity** → Avoid ambiguity
- **Specificity** → Define exact task
- **Context** → Provide background
- **Constraints** → Limit output (length, format, style) **Intent** → Clearly state what is required

Example:

- Weak: *"Explain AI"*
- Strong: *"Explain AI in 5 bullet points with real-world examples for beginners"*

iii. Prompt Patterns / Techniques The common patterns of the prompt along with their usage are as follows:

a) Role-Based Prompting

"You are an AI professor…"

b) Step-by-Step Prompting

"Explain step-by-step…"

c) Few-shot Prompting

Provide examples in the prompt

d) Instruction + Constraints

"Explain in 100 words using simple language"

e) Chain-of-Thought (Guided Reasoning)

"Show reasoning before final answer"

iv. Task-Oriented Prompting

Based on the tasks we can have different ways to provide the prompt. Following are the examples to design prompts for different tasks:

- **Learning/Explanation**
- **Coding**
- **Summarization**
- **Question generation**
- **Problem solving (AI, Cloud, etc.)**

v. Iterative Refinement (Very Important)

The first prompt may not be the best one. So, we must learn how the prompt can be refined to get most appropriate outcome:

- First prompt ≠ best prompt
- Improve by:
  - Adding constraints
  - Asking follow-up questions
  - Correcting output

Concept:

Prompt → Output → Refine → Better Output

vi. Evaluation of AI Output

It is very important to evaluate the output generated by AI. Critical thinking should be done to find the answer to the following:

- Is the answer **correct**?
- Is it **complete**?
- Is it **biased or vague**?
- Does it follow instructions?

vii. Ethical and Responsible Use

- Avoid Misuse and Plagiarism
  - Do not copy AI-generated content blindly for assignments or research
  - Always ensure originality and proper attribution when required
- Verify Information
  - AI tools like ChatGPT may generate incorrect or outdated information
  - Cross-check important facts before using them in academic or professional work
- Use AI as a Support Tool
  - AI should assist learning, not replace critical thinking
  - Students should understand concepts rather than depend entirely on AI-generated answers

viii. Basic Applications in Academics

- Generate notes
- Create MCQs and scenario-based questions
- Debug code

ix. Perform the following tasks:

- Convert a bad prompt → good prompt
- Generate:
  - Notes
  - Quiz questions
  - Code snippets
- Design a prompt for:

  Adversarial Search explanation

EXPECTED OUTPUT:

- Definition
- Game tree explanation
- Minimax working
- Alpha-Beta optimization
- Example
- Complexity analysis

INPUT & OUTPUT:
Design the prompts and provide the analysis using the following criteria:

Attempt	Prompt Quality	Output Quality	Improvements
1	Basic	Vague	Add structure
2	Improved	Better	Add constraints
3	Refined	High quality	Final

Attempt	Prompt Quality	Output Quality	Improvements
1	Basic	Vague, unstructured	Add clear sections (definition, example, algorithms)
2	Improved	More organized but moderate depth	Add constraints like step-by-step working, diagrams
3	Refined	High-quality, exam-ready	Final optimized prompt with structure, clarity, completeness

Explain adversarial search using Othello."

Adversarial search is a concept in AI used for decision-making in competitive environments where one player's gain is another player's loss (like board games). A classic example is the game **Othello** (also called Reversi).

🔹 What is Adversarial Search?

Adversarial search deals with:

- **Two or more players**
- **Opposite goals (competition)**
- Each player tries to **maximize their own advantage** and **minimize the opponent's**

In simple terms:
👉 *"I make the best move assuming my opponent will also make the best possible counter-move."*

🔹 Othello as an Example

In **Othello:**

- Two players: **Black** and **White**
- Players take turns placing discs
- The goal is to have the **maximum discs of your color** on the board at the end

Every move:

- Flips opponent's discs
- Changes the future possibilities of the game

This makes it a perfect example of adversarial search.

🔹 How Adversarial Search Works in Othello

1. Game Tree

- All possible moves form a **tree structure**
- Each node = a board state

- Your opponent is **perfectly rational**

✦ Get Plus ×

Example:

- You (Max) choose a move
- Opponent (Min) responds with the worst possible move for you
- You evaluate all outcomes and pick the best among the worst-case scenarios

---

3. Evaluation Function

Since exploring the full game tree is too large, we use a heuristic:

In Othello, evaluation may consider:

- Number of discs
- Corner positions (very important)
- Mobility (number of possible moves)

---

4. Alpha-Beta Pruning

Optimization of Minimax:

- Cuts off branches that won't affect the final decision
- Makes the search faster

🔹 Simple Flow in Othello

1. Generate all possible moves
2. Simulate opponent responses
3. Apply Minimax
4. Use evaluation function
5. Choose best move

🔹 Key Idea

👉 In Othello, adversarial search helps a player **plan ahead** by considering:

- "What will my opponent do next?"
- "How can I counter that?"

"Explain adversarial search using Othello. Include definition, game tree, minimax algorithm, alpha-beta pruning, and an example."

Here's a **clear, slightly detailed but easy-to-understand answer** covering all required parts:

🧠 Adversarial Search using Othello

🔹 1. Definition

Adversarial search is a search technique used in Artificial Intelligence for problems involving **competition between two players**, where:

- One player's gain = other player's loss
- Both players act **rationally and optimally**

👉 In simple words:
Each player tries to make the best move while assuming the opponent will also play perfectly.

🔹 2. Othello Game Overview

- Two players: **Black** and **White**
- Played on an **8×8 board**
- Players place discs to **capture opponent discs**
- Goal: **Maximize your discs** by the end of the game

🔹 3. Game Tree

A **game tree** represents all possible moves from the current state.

Structure:

- **Root node** → Current board position
- **Branches** → Possible moves
- **Nodes** → Resulting board states
- **Leaf nodes** → Final or evaluated positions

👉 In Othello:

Get Plus ×

4. Minimax Algorithm

The **Minimax algorithm** is used to choose the best move.

Concept:

- **Max player (AI)** → Maximizes score
- **Min player (Opponent)** → Minimizes score

Working:

1. Generate all possible moves
2. Expand the game tree to a certain depth
3. Evaluate leaf nodes using a heuristic
4. Propagate values upward:
   - Max chooses **maximum value**
   - Min chooses **minimum value**

Formula idea:

- Max node → `max(children)`
- Min node → `min(children)`

---

5. Alpha-Beta Pruning

Alpha-Beta pruning improves Minimax by eliminating unnecessary branches.

Key Terms:

- **Alpha (α)** → Best value for Max so far
- **Beta (β)** → Best value for Min so far

Condition:

- If **α ≥ β**, stop exploring that branch

👉 Benefit:

- Reduces computation
- Gives same result as Minimax but faster

---

6. Example (Simple Othello Scenario)

Suppose we evaluate moves up to depth 2:

```
        Root (Max)
       /         \
     A             B
   (Min)         (Min)
   /   \         /   \
  3     5       2     9
```

Step-by-step:

- Node A → Min chooses min(3, 5) = **3**
- Node B → Min chooses min(2, 9) = **2**
- Root → Max chooses max(3, 2) = **3**

👉 So, **Move A is selected**

With Alpha-Beta Pruning:

- While evaluating B:
  - If Min finds value **2**, and Max already has **3**
  - Since 2 < 3, remaining branch (9) can be **pruned**

👉 Saves time by skipping unnecessary evaluation

---

7. Evaluation Function in Othello

Since full search is not possible, we use heuristics like:

- Number of discs
- Corner occupancy (very important)
- Mobility (available moves)

format. Includ… on, (2) rules, (3) game tree, (4) minimax with step-by-step… Get Plus × …lpha-beta pruning with conditions, (6) a numerical example with tree diagram, and (7) complexity analysis. Ensure the answer is concise, well-organized, and suitable for a 10-mark question."

Here's a **concise, structured, exam-ready answer (10 marks):**

🧠 Adversarial Search using Othello

1. Definition

Adversarial search is an AI technique used in competitive environments where:

- Two players have **opposing goals**
- Each player assumes the opponent plays **optimally**

👉 It selects the best move by considering **all possible opponent responses.**

---

2. Rules of Othello

- Played between **two players:** Black and White
- Board size: **8 × 8 grid**
- Players alternately place discs
- A move must **capture opponent discs** by enclosing them
- Captured discs are flipped
- Game ends when no moves are possible
- Winner = player with **maximum discs**

---

3. Game Tree

- A **tree representation** of all possible moves
- **Root node** → Current state
- **Branches** → Legal moves
- **Levels alternate** between Max (AI) and Min (opponent)
- **Leaf nodes** → Final or evaluated states

👉 Due to large size, tree is explored up to a **limited** ↓ **pth**

4. Minimax Algorithm (Step-by-Step)

Used to determine the **optimal move.**

Steps:

1. Generate all possible moves
2. Expand game tree up to depth *d*
3. Apply **evaluation function** at leaf nodes
4. Backpropagate values:
   - **Max node** → choose maximum
   - **Min node** → choose minimum
5. Select move with **best outcome**

Idea:

- Max tries to **maximize score**
- Min tries to **minimize score**

✦ Get Plus ×

5. Alpha-Beta Pruning

Optimization of Minimax to reduce computation.

Parameters:

- **Alpha (α)** = best value for Max
- **Beta (β)** = best value for Min

Pruning Condition:

👉 **If α ≥ β → stop exploring that branch**

Benefits:

- Reduces number of nodes evaluated
- Same result as Minimax
- Faster decision-making

---

6. Numerical Example (Game Tree)

```
        Root (Max)
       /          \
    A (Min)       B (Min)
    /    \        /     \
   3      5      2       9
```

Step-by-step:

- Node A → min(3, 5) = **3**
- Node B → min(2, 9) = **2**
- Root → max(3, 2) = **3**

👉 **Best move = A**

With Alpha-Beta Pruning:

- While evaluating B:
  - First value = 2
  - Since Max already has 3 → **prune 9**

---

7. Complexity Analysis

- **Minimax Time Complexity:**
  👉 $O(b^d)$
  where b = branching factor, d = depth
- **Alpha-Beta Pruning:**
  👉 Best case: $O(b^{d/2})$
  👉 Worst case: $O(b^d)$
- **Space Complexity:**
  👉 $O(d)$ (depth-first recursion)

✅ Conclusion

Adversarial search in Othello:

- Uses **game trees** to explore moves
- Applies **Minimax** for optimal decisions
- Uses **Alpha-Beta pruning** for efficiency

👉 Enables intelligent decision-making in competitive games.

CONCLUSION:

_____

_____

DISCUSSION QUESTIONS:

1. What is prompt engineering?

   _____

   _____

   _____

   _____

2. Why is prompt refinement important?

   _____

   _____

   _____

   _____

3. What is Alpha-Beta pruning?

   _____

   _____

   _____

   _____

4. Difference between weak and strong prompts?

   _____

   _____

   _____

   _____

REFERENCES:

- https://learnpromptengineering.dev
- https://www.linkedin.com/pulse/40-prompt-engineering-resources-upskilling-spencer- taylor-xje0e/
- https://justprompting.in/